

Intro. Dev. - TD Révision

TD de révision

1 Les conditionnelles

1.1 Table de vérité

Donnez la table de vérité des expressions suivantes :

1. $(A \wedge B) \vee C \wedge A$
2. $(A \wedge B) \wedge (C \vee B)$
3. $((A \wedge C) \vee B) \wedge (A \vee (C \wedge B))$

1.2 Mention

Donnez le code permettant d'afficher la mention obtenue par une personne, avec la note de cette personne entrée par l'utilisateur. On considère que **Excellent** est ≥ 18 , **Très bien** est entre 16 et 18, **Bien** entre 14 et 16, **Assez bien** entre 12 et 14. On vérifie aussi que la note entrée est valide (entre 0 et 20).

1.3 Un exercice sur les années bissextiles (encore)

Donnez le code permettant de calculer si une année est bissextile. Pour rappel, les années bissextiles sont divisibles par 4, mais ne doivent pas être divisibles par 100 sauf si elles sont aussi divisibles par 400. Exemples : 1900 n'est pas bissextile car divisible par 100 mais pas par 400, 2000 est bissextile car divisible par 400, 2004 est bissextile car divisible par 4 et pas par 100, 1933 n'est pas bissextile car pas divisible par 4.

Pour plus de challenge, écrivez ce code dans une seule condition : un seul *if...else*.

1.4 Nombre de jours

Exercice complémentaire d'un TD : donnez un algorithme permettant de calculer le nombre de jours exact entre deux dates, en n'utilisant aucune boucle. (Attention, exercice difficile)

2 Les boucles

2.1 Épargne

Donnez le code permettant de calculer l'évolution d'une somme placée dans un compte d'épargne. L'algorithme demande à l'utilisateur de donner une somme initiale, un nombre d'années et un taux d'intérêt, et grâce à une boucle, il calcule ensuite l'argent que la personne a sur son compte s'il ne touche pas à l'argent pendant toutes ces années avec le taux d'intérêt appliqué à chaque année. Pour rappel, si le taux d'intérêt est de 5% et la somme initiale est de 1000 euros, la somme à la fin de la première année est de $1000 + 5\% \times 1000$, soit 1050. La somme à la fin de la deuxième année est de $1050 + 5\% \times 1050$, soit 1102,5.

Ajoutez maintenant l'algorithme permettant d'indiquer le nombre d'années à attendre pour dépasser une certaine somme. Par exemple : combien de temps pour dépasser les 10000 euros si l'on a 1000 euros initialement et un taux d'intérêt de 5%.

2.2 Base 2

En utilisant la concaténation et la chaîne de caractères (ex : $a = "1" + a$), produisez le code permettant de saisir un entier et de donner sa version en base 2. Pour rappel, il suffit de répéter la division entière par 2 et de concaténer les restes des divisions (voir le cours d'architecture des ordinateurs, internet, l'enseignant, votre voisin, ...).

2.3 Diviseurs

Donnez le code permettant d'afficher tous les diviseurs de tous les nombres inférieurs à une valeur donnée par l'utilisateur. Par exemple, si l'utilisateur entre 15, l'algorithme affiche tous les diviseurs de 1, de 2, de 3, de 4, ..., de 14 et de 15

3 Fonctions

3.1 Des fonctions, fallait que ce soit des fonctions !

Donner les sous-programmes suivants :

- Un sous-programme **saisir** permettant à l'utilisateur de saisir un entier. Avec `saisir(x)`, on doit permettre de modifier la valeur de la variable `x` avec une entrée de l'utilisateur
- Un sous-programme **min** qui retourne le min de deux valeurs données
- Un sous-programme **affiche** qui affiche la valeur donnée en paramètre

- Un sous-programme **estNote** qui retourne vrai si la valeur donnée en paramètre est une valeur entre 0 et 20
- Un sous-programme **minNote** qui demande à l'utilisateur de saisir en boucle des valeurs tant que l'utilisateur n'a pas entré de valeur en dehors de $[0, 20]$ et qui retourne la valeur la plus basse trouvée. Cette fonction doit évidemment utiliser toutes les fonctions précédentes.
- Donnez le main permettant d'afficher le résultat de **minNote**.

3.2 C'est l'heure de la récu-cu-cu-cu-cu-cursivité !

Donnez le sous-programme **fibonacci** qui prend un entier en paramètre en entier et qui retourne la valeur de la suite de Fibonacci de ce nombre. Le sous-programme doit être récursif, sans boucle, et seulement faire un appel récursif de **fibonacci**. Proposez aussi une version non itérative, et discutez de la différence entre les deux.

3.3 Help : j'ai les cosinus bouchés

Refaire l'exercice du contrôle de TP sur les cosinus :

Codez le sous-programme qui calcule le cosinus. Décomposez ce calcul en plus petits sous-programme : un sous-programme factoriel et un sous-programme puissance. Rappel : $\cos(x) = \sum_{n=0}^{+\infty} \frac{(-1)^n \times x^{2n}}{(2n)!}$. Puisque l'on ne peut évidemment pas faire une somme infinie sur ordinateur, la valeur maximum de n sera demandée en paramètre. Une version **récursive** et une **non récursive** (avec des boucles) sont attendues.

4 Les tableaux

4.1 Rotation de valeurs dans un tableau

Donnez le programme qui permet à l'utilisateur d'entrer un certain nombre de valeurs entières et qui stocke les valeurs dans un tableau. Puis ajoutez le programme permettant de décaler de x toutes les valeurs du tableau. Par exemple, le tableau $\{1, 2, 3, 4, 5\}$ avec un décalage $x = 2$ donne le tableau $\{4, 5, 1, 2, 3\}$. Un indice : on code le décalage d'une case, et on le répète x fois.

4.2 Mémoire d'un ordinateur

Donnez le sous-programme qui copie le fonctionnement d'une mémoire. On dispose d'un tableau de taille fixe pouvant stocker 100 entiers, chaque valeur entière étant codée sur 32 bits en base 2, c'est-à-dire que l'on dispose d'un tableau de tableaux de booléens. Donnez le programme permettant à l'utilisateur de saisir des entiers pour remplir ce tableau. Donnez le programme recherchant la valeur

maximale du tableau (à noter que les valeurs sont comprises dans l'intervalle $[0, 2^{16} - 1]$).

4.3 Dichotomie (chotomie)

On considère ici un tableau d'entiers qui est trié dans l'ordre croissant des valeurs. Produisez l'algorithme permettant de trouver une valeur dans cet ensemble, donnez aussi sa version dichotomie. Vous pouvez aussi faire la version récursif de la dichotomie.

5 Les pointeurs

5.1 Pointeur qui pointe

Donnez le sous-programme qui prend en paramètre un tableau et deux pointeur (min et max). Le sous-programme fait ensuite pointer les pointeurs sur la valeur min et la valeur max du tableau.

5.2

(Le titre de l'exercice pointe sur NULL)

Donnez l'algorithme permettant de créer une matrice (un tableau de tableaux) où chaque case est un entier. La première ligne de la matrice à 10 cases, avec des valeurs aléatoires dans $[1, 10]$ dans chaque case. Il y a 10 lignes et chaque ligne (sauf la première) possède n cases où n est égal à la dernière valeur de la ligne précédente. Exemple :

```
4 5 8 1 3 4 5 4 8 9
4 1 3 6 4 7 1 5 4
1 4 9 2
6 6
1 8 7 9 6 2
7 1
1
4
7 8 5 4
4 7 8 5
```

Affichez les adresses mémoires des valeurs entières stockées. Quel est la taille mémoire d'un entier dans votre ordinateur (1 case mémoire = 8 bits) ?

5.3 *Amour, gloire et pointeurs Cinematic Universe*

Amour, gloire et pointeurs connaît un énorme succès et les producteurs ont décidé d'étendre le concept et d'en faire plusieurs séries (le *Amour, gloire et pointeurs Cinematic Universe*). L'objectif de cet exercice est de réfléchir à

une structure de donnée permettant d'associer des personnes (réalisation, production, équipe de tournage,...) et des épisodes. On considère qu'une personne peut intervenir sur plusieurs épisodes, et un épisode est représenté par un enregistrement/structure nommé *episode*, dont le contenu n'est pas important pour cet exercice. A vous de proposer la structure des données (avec les sous-programmes) permettant de :

- Ajouter/retirer facilement un épisode de la liste des épisodes affectés à une personne.
- Créer un épisode et l'associer directement à une personne. On considère que le sous-programme void remplirInformationsEpisode(*episode* ep) s'occupe de demander à l'utilisateur de rentrer des valeurs pour remplir les attributs de ep, mais attention : ce sous-programme ne crée pas la variable.
- Vérifier si deux personnes travaillent sur un épisode en commun
- Soit la fonction float accesValeur(*episode* ep) qui retourne une valeur liée à ep (ex : heures de tournage, coût,...). Calculer la somme des accesValeur de tous les épisodes où intervient une personne.
- Supprimer tous les épisodes.

Voici des éléments à prendre en compte:

- La modification d'une caractéristique d'un épisode doit être rapide, et ne doit pas nécessiter un parcours de toutes les personnes intervenant dans cet épisode.
- On dispose de N personnes (N ne varie pas) et l'on sait sur combien d'épisodes maximum peut intervenir chaque personne. On dispose donc de nombreEpisode(int p) qui retourne le nombre d'épisodes sur lesquelles peut travailler au maximum la personne p
- Lorsqu'un épisode n'a plus personne travaillant dessus, il est automatiquement supprimé. Pour cela, on considère que ep.length() renvoie le nombre de pointeurs pointant vers ep.

Bien évidemment, on souhaite utiliser des pointeurs et chaque personne possède plusieurs pointeurs vers les épisodes dans lesquels elle intervient.

6 Liste chaînée

6.1 Libéréééééé, Liste Chaînéééééé

Donnez les algorithmes des primitives des listes chaînées de float. Pour chaque algorithme, déroulez à la main son fonctionnement sur une ou plusieurs listes chaînées (notamment une liste vide). Prendre du temps, dérouler l'algorithme sans omettre d'étape est un moyen pertinent de comprendre pourquoi et comment on fait.

6.2 Un exercice qui est dans les choux, et qui tombe dans les pommes

Codez le jeu du chou-pomme avec une liste doublement chaînée, utilisant un pointeur tête et un pointeur queue, dont les primitives sont données (pas besoin de les recoder, ni de recoder la liste). Dans le jeu du chou-pomme, deux joueurs s'affrontent en avançant à tour de rôle d'un pas, le gagnant étant celui qui marche sur le pied de l'autre en premier. Avec la liste chaînée, il faut que chaque joueur commence à une extrémité et à tour de rôle, un joueur peut avancer de 1,2 ou 3 maillons. Codez les fonctions permettant de jouer à ce jeu (initialisation aléatoire, tour de chaque joueur, condition de victoire,...).

6.3 C'est pas parc'que you are me, Qu'I am you

Le but de cet exercice est de coder une structure de données représentant un texte où l'on peut facilement modifier chaque mot par un synonyme, une traduction, ou un autre mot correspondant (voir exemple). On peut ainsi créer des phrases aléatoires à partir d'un modèle. Pour cette exercice, on utilise une liste chaînée pour représenter un texte, chaque maillon étant une liste chaînée **cyclique**, où chaque sous-maillon est un synonyme (ou une traduction). Une fois le texte initialisé, on peut produire un texte exprimant la même chose mais avec des mots différents (pour éviter le plagiat) juste en "tournant" la roulette de chaque mot (la liste cyclique). Exemple :

Mot numéro	1	2	3	4	5	6	7	8	9	10	11
Phrase →	Tu You Vous Usted Ils	pleurais pleures cry chialaed geins	a one pareillement comme similarly similar	fraise pomme deux une nueve un	apple madeleine cookie donut brownie	Pas Not	tengo ai lui me moi Yo Je	I	avoir have je	my	dignité un chat

Dans cet exemple, on dispose d'une liste chaînée de 11 maillon, chaque maillon correspond à un ou plusieurs mots (stockés dans une liste chaînée cyclique). On peut changer la phrase en "tournant" le cycle de chaque mot, par exemple avec trois rotation sur le premier mot, la phrase devient : Ils chialaed comme une madeleine Not me I have my dignité. Pour coder cet exercice, vous pouvez recoder les deux types de liste chaînée ou considérer que ces listes sont déjà définies. Codez les sous-programmes permettant de : ajouter un mot (et ses synonymes) à la phrase, générer une nouvelle phrase aléatoire (en changeant chaque mot par un synonyme), supprimer un mot (et ses synonymes).

7 Pile et file

7.1 Navigateur internet

Dans cet exercice, on considère que les structures de pile et de file sont données. Codez la mise en place des boutons “retour en arrière” et “retour en avant” d’un navigateur internet. Une adresse d’un site internet est simplement une chaîne de caractères.

7.2 Cet exercice de Fibonacci est aussi mauvais que la combinaison des deux derniers

Dans cet exercice, on simule un processeur 4 coeurs, chaque coeur exécutant des processus stockés dans sa file avec une taille limite de 3 processus. Les processus ici sont simplement des sous-programmes à exécuter (représenté par une structure *appel* donnée), et chaque processus nécessite 5 unités de temps (5 instructions) pour être exécuté. On dispose d’une pile pour lister les appels récurrents des sous-programmes. Codez la mise en place de cette structure de données tel que : une pile stocke des *appels*, chaque coeur dispose d’une file d’*appels*, une fonction permet d’envoyer un *appel* de la pile vers le coeur avec le moins d’*appels* dans sa liste, une fonction permet de vérifier qu’aucun coeur est inactif, une fonction permet d’exécuter une instruction dans chaque processeur. On applique ce problème à l’exécution de la suite de Fibonacci en version pile, où $\text{Fibonacci}(n)$ est une fonction retournant un *appel* à la fonction Fibonacci de n .

7.3 Files comme le vent Pile-Poil

Implémenter les sous-programmes `pileTaille`, `fileTaille`, `inversePile`,... mais en utilisant le plus possible la récursivité.

8 Algorithmes de tri

8.1 Appliquer des algorithmes de tri

Triez les tableaux suivants avec les algorithmes de tri vus en cours :

2	4	1	12	6	8	10	3	5	9	11	7
11	9	3	5	7	1	2	4	12	8	10	6

8.2 Le tri du singe

Le tri stupide, ou tri du singe ou bogosort, est un tri très stupide, vraiment très stupide. L’algorithme mélange juste aléatoirement le tableau jusqu’à ce que le tableau soit trié. Décrivez la complexité pire cas, cas optimal, et cas moyen. Implémentez l’algorithme. Des idées d’algorithmes encore plus stupides ?

8.3 Tri stable

Un tri stable est un tri qui conserve l'ordonnancement initial des éléments que l'ordre considère comme égaux. Pour chaque algorithme vu en cours, exprimez s'il est stable ou non

9 Complexité

9.1 Un exercice sur les complexités (pikachu_surpris.jpg)

Donnez les complexités des morceaux de codes suivants :

1.

```
for (int i = 0; i < n; ++i) {  
    Afficher(" Hello World !");  
}  
for (int j = 0; j < n; ++j) {  
    Afficher(" Hello World !");  
}
```
2.

```
for (int i = 0; i < n; ++i) {  
    Afficher(" Hello World !");  
}  
for (int j = 0; j < 1000000000; ++j) {  
    Afficher(" Hello World !");  
}
```
3.

```
for (int i = 0; i < n; ++i) {  
    Afficher(" Hello World !");  
}  
for (int j = 0; j < m; ++j) {  
    Afficher(" Hello World !");  
}
```
4.

```
for (int i = 0; i < n; ++i) {  
    for (int j = 0; j < n; ++j) {  
        Afficher(" Hello World !");  
    }  
}
```
5.

```
for (int i = 0; i < n/2; ++i) {  
    for (int j = 0; j < n/2; ++j) {  
        Afficher(" Hello World !");  
    }  
}
```

6. for (int i = 0; i < 1000000; ++i) {
 for (int j = 0; j < 1000000; ++j) {
 Afficher("Hello World !");
 }
 }
7. for (int i = 0; i < n*n; ++i) {
 Afficher("Hello World !");
 }
8. for (int i = 0; i < n; ++i) {
 for (int j = 0; j < m; ++j) {
 Afficher("Hello World !");
 }
 }
9. if (i == 0) {
 for (int j = 0; j < n; ++j) {
 Afficher("Hello World !");
 }
 }
10. for (int i = 0; i < n; ++i) {
 if (i < 0) {
 for (int j = 0; j < n; ++j) {
 Afficher("Hello World !");
 }
 }
 }

9.2 Les complexités font mal à la tête

Proposez les algorithmes suivants :

1. Un algorithme en $O(n!)$
2. Un algorithme en $O(1)$ pour trier un tableau dont la taille est inférieure à 100 éléments
3. Un algorithme dont la complexité est infinie
4. Un algorithme en $O(n)$ mais dont le meilleur cas est en $O(1)$
5. Un algorithme en $O(1)$ et un algorithme en $O(n^2)$ qui font exactement la même chose (tant que n est inférieur à un nombre très grand) mais où l'algorithme en $O(n^2)$ est plus performant que $O(1)$

6. Un algorithme avec une complexité pire que $O(n!)$ mais sans que ce soit $O(\infty)$
7. Un algorithme avec une complexité $O(\infty)$, sans que ce soit une boucle infinie

9.3 : l'exercice 9.3

Donnez les complexités des algorithmes de toutes les questions de ce TD.